

MSSE Capstone Project

Sprint 1 Progress Stand-Up & Sprint Recovery, Integration and Deployment Alignment

Meeting Minutes

TrustAI Marketplace

Meeting 1 Date	July 22, 2026
Meeting 1 Time	Approximately 7:02 PM CAT / 10:02 AM PDT
Meeting 1 Duration	Approximately 9 minutes 59 seconds
Meeting 2 Date	August 6, 2026
Meeting 2 Time	Approximately 7:00 PM CAT / 10:00 AM PDT
Meeting 2 Duration	Approximately 36 minutes
Location	Virtual team meetings
Meeting Types	Sprint 1 progress stand-up; later sprint recovery/planning, architecture review, integration and deployment alignment
Prepared For	TrustAI Marketplace MSSE Capstone team records

1. Attendees and Working Roles

The same five-member TrustAI Marketplace team continued to carry the project through these meetings. The conversations also show how each person's working role had become clearer by this stage: Ranga was driving much of the backend and architectural discipline; Mulima was keeping the sprint and Trello process together; Adrian was leading the frontend and visual implementation; Samar was increasingly focused on DevOps and production deployment; and Ahmed was concentrating on product direction, AI behavior, integration, testing, and the final public-facing release.

Team member	Working role reflected in the meetings
Ahmed Al-Mandalawi	Product Owner / AI Analysis Lead. Continued to focus on end-to-end functionality, production integration and testing, AI-output quality, documentation, and the custom-domain connection once the production environment is available.
Ranga Nyamadzawo	Backend/platform contributor. Led database/schema work, architecture alignment, code/PR discipline, shared-contract guardrails, and support for CI/CD and deployment.
Mulima Chibuye	Scrum Master / Project Manager. Managed sprint recovery, Trello status and ownership, planning discussions, and chaired the August 6 meeting.

Team member	Working role reflected in the meetings
Adrian Muchatibaya	Frontend lead and wireframe owner. Continued React implementation, frontend/backend integration, and alignment to the previously agreed visual artifacts.
Samar El Ghandour	QA / DevOps lead and backend contributor. Investigated and began implementing the AWS production deployment path using GitHub Actions, ECR and EC2.

Speaker-attribution note: The July 22 transcript briefly refers to a person as 'Dave' while asking for an LLM-status update. The active project roster in the surrounding team records contains no member by that name, so this document does not silently assign that update to another person.

2. Combined Executive Summary

Taken together, the two meetings capture an important transition in the TrustAI Marketplace project. The July 22 meeting still felt like an implementation stand-up: the team was getting the database schema and migrations into place, bringing login functionality online, aligning frontend work to the agreed wireframes, and putting guardrails around implementation work so that generated changes would not quietly break shared contracts. The team already knew it was behind, but the main concern was still how to build the remaining pieces correctly and keep everyone coordinated.

By August 6, the tone had changed. The application had advanced far enough that team members were testing it locally and talking seriously about production. Many pull requests had been closed, but the project was still behind the Trello plan, and the bottleneck had shifted from basic implementation to integration, review latency, architecture alignment, deployment, and release readiness. The team recognized that a queue of approved-but-unmerged pull requests was creating unnecessary conflicts and that decisions recorded in PR #34, the codebase, the documentation, and the Trello board all needed to tell the same story.

The largest August 6 discussion concerned deployment. Render had been considered earlier because of its simplicity, and a skeleton frontend had already been deployed there, but the team was now leaning toward AWS. Samar had experimented first with cloning the repository directly onto EC2 and then proposed a more controlled image-based path using Amazon ECR. The group verbally aligned on a workflow in which GitHub Actions builds the Docker image, authenticates to AWS through OIDC, pushes the image to ECR, and deploys it to EC2. Samar retained ownership of that pipeline, Ranga offered to assist and review deployment PRs quickly, and Ahmed emphasized that the environment must be available early enough for the custom domain, production integration testing, AI validation, and final polish.

A recurring lesson across both meetings was that the project could no longer afford hidden work or informal assumptions. Trello cards needed real owners and accurate statuses; pull requests needed prompt reviews and merges; architecture changes needed to be documented as they happened; and testing needed to move from isolated local checks toward a stable production workflow. The group therefore agreed to fast-track remaining work, overlap independent tasks where safe, and protect time for deployment, regression testing, documentation, and the final presentation.

3. July 22, 2026 - Sprint 1 Progress Stand-Up

3.1 Meeting purpose and immediate context

The July 22 recording is a short, highly practical stand-up. Rather than revisiting the full architecture, the team concentrated on what had actually moved since the July 15 alignment meeting and what still needed attention during Sprint 1. The discussion touched the backend schema and migration work, login/front-end progress, the still-pending LLM/provider decision, frontend fidelity to Adrian's wireframes, the use of Claude Code, regression protection, and the day-to-day Trello habits needed to keep several people working without duplication.

The schedule problem was already visible. One participant initially questioned whether the project was truly behind, but the group quickly agreed that it was. The phrase 'trying to play catch up' accurately captured the working mood: enough foundations were finally in place for faster development, but the team needed the current sprint to move more quickly if it wanted a substantially working application by the next retrospective.

3.2 Database schema and Alembic migration work

The meeting opened with a screen share of the backend data work. The presenter had already created and pushed the schema and was in the process of preparing the pull request description. The implementation was deliberately based on the entity relationships and contracts the team had already defined rather than on a new data model created in isolation.

- The database schema and its columns had been created and pushed.
- Alembic was selected as the migration tool because it works naturally with SQLAlchemy, which was already the project's ORM layer.
- The schema followed the entity relationships already expressed in the project's contracts.
- Comments were added to the implementation to explain why particular structures existed and what project decisions they were following.
- The pull request was expected to document the schema, the reasoning behind it, and the relevant security/guardrail considerations.
- The presenter reported no active impediment and expected to move immediately to the next backend ticket after review.

This was an important sign that the project was moving beyond a static schema design into a maintainable application database. Using migrations meant the team could evolve the schema through reviewed changes rather than relying on ad hoc manual database updates.

3.3 Login and frontend progress

The next update came from the frontend side. Progress had started slowly, but a second pull request was expected to introduce working login functionality. The newly available schema was seen as useful because the team could now test more of the application against a realistic backend and database rather than validating screens or routes separately.

The team wanted the next review to show meaningful, visible progress. The expectation was no longer simply that individual tickets would move; the team wanted something closer to a functioning application that could be exercised from the user interface through to the backend and persistence layer.

Schedule concern: The team explicitly agreed that it was behind and needed to fast-track the current sprint. The next retrospective was expected to show a significantly more complete and demonstrable application.

3.4 LLM/provider decision remained open

The LLM/provider strategy was still a live dependency. A card existed for evaluating the provider choice, and the team tried to obtain an update during the call. The person being addressed could not be heard, so the group decided to follow up after the meeting rather than spend the short stand-up waiting. No new provider decision was recorded on July 22.

3.5 Frontend implementation had to follow the agreed wireframes

Ranga asked whether the frontend work had been checked against the wireframes and mockups Adrian had previously created. The response was candid: the current work had referenced the wireframes, but the developer had been concentrating first on getting core functionality working and had not yet performed a full visual comparison.

The team nevertheless treated the wireframes as agreed design artifacts, not optional inspiration. The frontend would be brought into strict alignment with them as implementation progressed. This preserved the earlier decision that even though multiple people might touch the frontend, the finished application needed one consistent visual language.

3.6 Repository-Level Development Guardrails

The conversation then moved to repository-level development guardrails. The team discussed maintaining project-specific guidance in the repository so implementation work would remain within the same architectural rules, code conventions, contracts, and regression expectations agreed by the team.

- The file should describe the project's agreed design philosophy, code conventions, contracts, and implementation guardrails.
- Team members should be free to read and improve those rules so the file represents the project rather than one developer's prompt.
- Implementation changes should not stray from the contracts that had already been agreed.
- New changes should be checked for simple regressions so that work added to solve one ticket does not quietly damage behavior already merged.
- Schemas, shared contracts, and API contracts were called out as especially sensitive areas.
- If solving a ticket appears to require changing one of those shared contract areas, the change should not be made silently; it should be reviewed explicitly by the relevant owners.

The significance of this discussion was the attempt to convert architecture and contract decisions into repeatable development discipline. The same concern later resurfaced more formally in the August architecture and pull-request discussion.

3.7 Adrian's frontend work and avoiding branch conflicts

Adrian reported that he was continuing to work on the frontend using the original wireframes. He had been slowed during the previous week because he was out of office, but expected to push his changes when ready.

He also described some uncertainty about overlapping work and whether he should hold back to avoid conflicts. The response was that he should continue on a separate appropriate branch or task and merge through the normal pull-request process. The point was not to stop parallel development; it was to make the overlap visible and coordinate it properly.

3.8 Trello ownership and asynchronous working discipline

The meeting ended by returning to project tracking. The team wanted Trello to be reliable enough that people could understand the current state of the sprint without waiting for another call.

- As soon as someone picks up a sprint or backlog item, the card should show that person's ownership.
- The card should be moved into the correct workflow column immediately.
- Visible assignment should prevent two people from unknowingly implementing the same feature.
- Accurate status makes asynchronous work possible across time zones and reduces the need for synchronization meetings purely to discover who is doing what.

This was not a new rule, but July 22 reinforced it because work was beginning to overlap. The August 6 meeting later showed that this discipline still had not been applied consistently enough.

3.9 July 22 decisions and working agreements

ID	Decision / working direction	Why it mattered
J22-01	Use Alembic for database migrations with SQLAlchemy.	Gives the project a controlled, reviewable way to evolve the PostgreSQL schema.
J22-02	Keep frontend implementation aligned with Adrian's approved wireframes and mockups.	Preserves design consistency while multiple contributors implement the UI.
J22-03	Add repository-level development guardrails.	Turns architecture and coding rules into shared instructions for AI-assisted development.
J22-04	Protect schemas and API contracts from silent opportunistic changes.	Shared contracts are the basis for parallel work and require human review when they change.
J22-05	Use regression awareness as part of implementation changes and PR review.	New work must not break behavior already merged into the project.
J22-06	Assign and move Trello cards as soon as work starts.	Prevents duplicate work and supports asynchronous coordination.
J22-07	Accelerate the current sprint.	The team recognized that it was already behind and needed a working application quickly.

3.10 July 22 action items

Owner	Action	Status / target
Backend/schema owner	Finish the schema/migration PR description, request review, and then continue to the next backend ticket.	Immediate / in progress
Frontend contributors	Complete login functionality and begin fuller testing against the working backend and database.	Current sprint
LLM work owner	Provide the missing update on the provider/strategy card through the team channel.	Follow-up required

Owner	Action	Status / target
Ranga / maintainers	Add repository development guidance and document the project guardrails, protected contract areas, and regression expectations.	Planned in current PR/workstream
Adrian	Continue frontend implementation from the agreed wireframes and raise PRs as work is completed.	In progress
All members	Assign themselves to Trello cards immediately and keep each card in the correct workflow column.	Ongoing

4. Progression from July 22 to August 6

The two meetings are especially useful when read together because several concerns raised on July 22 became concrete integration problems by August 6.

- The July discussion about protecting shared contracts evolved into a broader architecture-alignment issue documented through PR #34.
- The July concern about regression protection became more urgent when delayed PR merges began causing conflicts with newer changes already in main.
- The July rule that Trello cards should show ownership and status was still not being followed consistently enough by August; the board itself needed cleanup.
- The application moved from schema/login foundations to a locally functioning end-to-end product, making production deployment and integration testing the next major risk.
- The deployment question matured from earlier Render experiments into a proposed AWS image-based workflow using GitHub Actions, OIDC, ECR, and EC2.
- The schedule moved from 'we need to catch up' to an explicit recovery strategy in which sprint work would be fast-tracked and, where safe, performed in parallel to protect time for release activities.

5. August 6, 2026 - Sprint Recovery, Integration and Deployment Alignment

5.1 Informal opening and competing academic workload

The August 6 call began informally with a discussion of the final two MSSE courses/electives and the August 12 deadline. Several members still had coursework outstanding and described the remaining modules as bulky and time-consuming. Although this was not a project decision, it was relevant to capacity: the capstone was entering its final weeks at the same time that members were trying to finish other program requirements.

The opening also reflected a broader observation about team development. Individual contributors might be able to move quickly on their own, but a shared project depends on coordination, reviews, responses, and sequencing. The team agreed that communication was therefore one of the things determining how quickly the capstone could move.

5.2 Missed sprint ceremony and meeting chair

When the full group was present, the team discussed who would chair. Ahmed was suggested, but he explained that he was already taking notes and intended to prepare the meeting minutes. Mulima then took the chair.

The team treated the call as a replacement for the sprint retrospective/planning meeting that had been missed. Rather than dwell on the missed ceremony, Mulima proposed a practical agenda: review what had been completed, identify what was still pending, understand why the team was behind, and decide how to get back on track.

5.3 Trello status showed the project was behind

Mulima reviewed the Trello board and stated directly that the team was behind plan. The board showed seven cards in progress and eleven cards in the current sprint, with those current-sprint items ideally due by the following Wednesday. After that, another fourteen cards were expected to run through approximately August 26, followed by a further group of seven cards.

The transcript does not capture the final date attached to that last group, but the overall point was clear: a substantial amount of work remained and there was little room for slow reviews, stale branches, or duplicated effort.

Recovery priority: Reduce the amount of work sitting in review or in ambiguous board states, complete the current must-have items quickly, and preserve enough time for deployment, production testing, documentation, and presentation preparation.

5.4 Ranga's capacity update and the need for team feedback on architecture

Ranga went first with a status update and apologized for being slower over the previous week and a half because production work in his day job had consumed most of his available time. He expected to be able to move faster during the coming week, but said his main blocker was the pull request containing his architecture review and recommendations.

His concern was not that every suggestion had to be accepted. He wanted a visible team response - comments, amendments, approval, or at least a clear acknowledgement - because the architectural decisions affected downstream work. He reminded the team that the project was collaborative and that the final recorded presentation would require every member to understand what had been built and why. The PR discussion itself therefore formed part of the material the team would later need to explain the system.

5.5 Approved PRs should be merged promptly

Ranga then raised a second process issue: reviewers were approving pull requests but sometimes leaving them open. That delay was beginning to create real cost. On the day of the meeting, he had needed to perform a second-pass review and make adjustments before merging other work because main had moved and the older approved branches had begun to conflict.

- If a reviewer approves a PR and has sufficient authority to merge it, the default should be to merge it rather than leave it open indefinitely.
- If a reviewer wants another pair of eyes, that need should be stated explicitly in WhatsApp so another team member knows action is required.
- Allowing many approved PRs to accumulate creates a risk that they will all be merged in a short period, producing a cluster of regressions at once.

- Early merging and early regression detection is cheaper than discovering integration failures near the end of the project.

The team broadly agreed. The discussion marked a shift from thinking of pull requests merely as a review mechanism to treating merge cadence itself as part of release risk management.

5.6 PR #34 and the architectural conflicts it documented

Mulima identified the PR under discussion as PR #34 and indicated that, from his perspective, it was already essentially complete. He said he would merge/close it during the meeting.

A previous comment on the PR was also clarified. Mulima had asked about removing or resolving conflicting branches/items; Ranga had interpreted the question as referring to the critical conflicts documented in the PR summary. Mulima confirmed that this was what he meant and observed that most of those conflicts appeared to have been resolved by then.

The practical outcome was that PR #34 was treated as the team's architecture-alignment record rather than an optional documentation exercise. Its decisions needed to flow into future implementation and into Trello so that planning artifacts did not continue to describe a different system from the one being built.

5.7 Local application status and the remaining History problem

Ahmed reported that he had tested the application repeatedly on his local machine and that, overall, it looked good. The frontend and backend were generally working together and the application felt substantially more complete than it had in July.

The important exception was History. The History endpoint/flow was not yet fully connected or implemented between frontend and backend. This mattered because saved analysis history was part of the authenticated user experience and because it was the clearest remaining break in an otherwise working local end-to-end flow.

Ahmed used that result to argue for an early deployment. A local application can appear stable while a public production environment exposes configuration, networking, environment-variable, domain, API, model, and persistence problems that do not appear locally.

5.8 Why deployment needed to begin before the final weeks

Ahmed and other members pushed for deployment to happen with enough lead time for debugging and polish. The concern was especially strong around the AI behavior: the final presentation would be judged on whether the product actually generated useful analysis and displayed the correct result to the user, not simply on whether the interface looked complete.

The team therefore treated deployment as part of integration, not as a final packaging step. Once a public environment existed, the application could be exercised in the same conditions that would be used for the capstone demonstration.

5.9 Revisiting Render

Render remained part of the conversation because it had been suggested earlier and a skeleton frontend had already been deployed there. Its attraction was simplicity: it was easier to get a web application online quickly than building a full infrastructure workflow from scratch.

At the same time, cost and billing created hesitation. Ahmed described being surprised by charges on a previous project and noted that the service had eventually been suspended after consuming resources. The conversation did not conclude that Render was technically unsuitable; rather, the team was trying to decide whether a lower-cost AWS route could provide enough stability without introducing too much infrastructure overhead.

5.10 Ranga's cloud-provider position and Docker Hub contingency

Ranga said he was not committed to a particular provider. He was comfortable with Render or AWS as long as the solution was practical for the capstone. His main reservation was unnecessary cost: because this was a small academic project, he preferred to stay within free or very low-cost usage if possible.

He reasoned that a new AWS account might provide enough free-tier capacity for the review period. He also questioned whether ECR itself would be free and suggested Docker Hub as a possible fallback registry if ECR cost became problematic. The idea was that the deployment could still remain image-based even if the image registry changed.

5.11 Samar's original EC2 experiment

Samar explained that she had already built part of the AWS path. Her first experiment used a straightforward approach: clone the GitHub repository directly onto an EC2 instance, install/run Docker there, and deploy from the server.

That approach proved useful as a proof of concept, but she was now investigating a more secure production workflow. She wanted the runtime server to receive a deployable container image instead of functioning as a source-code checkout and build machine.

5.12 ECR as the intermediary between GitHub and EC2

Samar proposed Amazon ECR as the intermediary. She was still learning and experimenting with the exact implementation and described herself as approximately halfway through the work, but she believed this was the best direction.

Her explanation was that the registry would create a cleaner and more controlled channel between the build process and the EC2 runtime. Rather than storing the repository and build state on the production machine, the deployment would move a prepared image through ECR and let EC2 run that artifact.

The team accepted the high-level idea. The discussion then became more specific and translated Samar's concept into the CI/CD sequence the group wanted.

5.13 GitHub Actions -> OIDC -> ECR -> EC2 deployment flow

The proposed production path was restated step by step during the call and verbally confirmed:

1. Changes are merged through the normal GitHub pull-request and review process.
2. GitHub Actions runs the CI/deployment workflow.
3. The workflow builds the Docker image for the application.
4. GitHub authenticates to AWS using OIDC rather than relying on a long-lived credential or leaving a deploy key on the production server.
5. The image is pushed to Amazon ECR.
6. The EC2 server retrieves/deploys the approved image from ECR.
7. Production environment variables and runtime configuration are applied.
8. Ahmed connects the custom domain once the public environment is ready.
9. The team performs production integration and smoke testing against the actual public deployment.

Deployment direction agreed in the meeting: Use GitHub Actions to build and deliver the Docker image through AWS ECR to EC2, with OIDC for GitHub-to-AWS authentication. Samar drives the pipeline; Ranga supports/reviews; Ahmed follows with the domain and production testing.

5.14 Free-tier discussion and cost uncertainty

The group briefly discussed how long the AWS free allocation would last. Different values were mentioned in quick succession - 30 days, one year, and six months - before the conversation settled on the view that six months would be enough for the capstone review window.

Because the meeting did not verify the current AWS pricing rules, the free-tier duration should be treated as a planning assumption from the meeting rather than a confirmed technical fact. The practical decision was simply that the team expected the project could remain online long enough for grading and could be shut down later before meaningful charges accumulated.

5.15 Integration and testing as Ahmed's release responsibility

Ahmed explicitly connected the deployment schedule to his integration/testing responsibility. He explained that if the environment arrived only at the very end, he would not have enough time to polish the application properly.

He placed functionality ahead of cosmetics: the critical requirement was that the AI generate the right type of analysis, that the backend return the correct information, and that the frontend render those results reliably. Those are the behaviors the team would ultimately demonstrate to evaluators.

The team accepted the point and agreed that upstream work needed to move faster so Ahmed could begin production testing sooner.

5.16 Sprint fast-tracking and overlapping work

The team discussed using the remaining schedule more aggressively. Rather than waiting for the current sprint to end before beginning every item in the next sprint, independent work could be brought forward where it would not create conflicts.

The aim was to complete the eleven current items while also beginning suitable work from the next group, thereby creating a larger window for deployment, integration, debugging, and final documentation. This was framed as schedule recovery rather than as an expansion of scope.

5.17 Adrian's deployment/frontend integration comments

Adrian said the AWS approach Samar described sounded feasible. He also reminded the group that a skeleton frontend had already been deployed on Render and that frontend/backend integration could continue as routes became available.

There was a short exchange about recent frontend pull requests and the home/front page. Ranga asked for the existing Render URL to be posted in the WhatsApp group so he could review it. That deployment remained useful as a test artifact even though the team was now treating AWS as the intended production path.

5.18 Sprint Demo Evidence and Submission Priorities

The team revisited whether separate sprint-demo recordings were required as part of the capstone evidence. The requirement was not confirmed during the meeting, so the group did not treat additional demo recordings as a priority over implementation, testing, documentation, deployment, and other assessed deliverables.

The working decision was to verify the formal submission requirement before allocating scarce project time to additional recording work. If separate sprint-demo evidence was required, the team would address it; otherwise, release readiness and the assessed deliverables would remain the priority.

5.19 Documentation is part of the product record

Documentation became another explicit release concern. One member noted that the final evaluation would look closely at the written evidence of the project and reminded everyone to document what they were doing.

Ranga distinguished between two kinds of documentation. User-facing instructions could be finished later, once the application behavior was stable enough to document accurately. Technical and project documentation, however, needed to be written continuously. If a change required an ADR, architecture review, design note, or other explanation, that record should be added to the repository's docs area in the same period as the change itself.

This matched the project's broader philosophy: the final report should be supported by evidence created during development rather than reconstructed from memory during submission week.

5.20 Trello had drifted from the real project state

Ranga asked Mulima whether the Trello cards had been updated to reflect the decisions and corrections raised in PR #34. Mulima confirmed that they had not yet been fully updated and agreed that the cleanup needed to happen.

- The three cards specifically referenced by PR #34 needed to be rewritten or updated to match the decisions the team had actually made.
- PR comments should reference the related Trello card whenever possible so implementation history can be traced back to planning.
- Some cards appeared to be completed but did not identify who had worked on them.
- Some completed work had not been moved into the Done column.
- The risk-gauge component was mentioned as one example where the board did not clearly reflect the real state of the work.
- Mulima committed to going through the relevant cards during the week and correcting ownership and status.

The team repeated the working rule from July 22: if someone starts a card, assign it and move it to In Progress; if the work is finished, move it to Done. The board should allow another engineer to understand what remains without starting duplicate work.

5.21 Immediate deployment ownership and support

By the end of the meeting, the deployment ownership was clear. Samar would continue the AWS pipeline work. Ranga noted that the existing GitHub Actions workflow already handled the first quality gates before code entered main but now needed to be extended to include the actual deployment.

Ranga offered to help Samar directly and asked her to tag him on deployment PRs so he could review quickly. Samar said she was dedicating the next several days to the project and expected the deployment work to become ready very soon.

Ahmed emphasized that this timing mattered because he could begin the domain and production-testing work as soon as the environment existed; he did not need to wait until every other project activity was complete.

5.22 August 6 decisions and working agreements

ID	Decision / direction	Reason / follow-up
A06-01	Treat PR #34 as the architecture-alignment record and move it toward closure/merge.	The team needs a shared explanation of what is being built and why.

ID	Decision / direction	Reason / follow-up
A06-02	Merge approved PRs promptly.	Prevents stale branches, avoidable conflicts, and a late burst of regressions.
A06-03	Use AWS as the intended production path, with GitHub Actions -> OIDC -> ECR -> EC2.	Creates an automated image-based deployment rather than cloning the repository directly onto production.
A06-04	Samar owns the deployment pipeline, with Ranga available to support and review.	Deployment is now on the critical path and needs quick feedback.
A06-05	Ahmed connects the custom domain and drives production integration testing once the environment is ready.	The final product must be proven in the public environment, not only locally.
A06-06	Begin production deployment and testing early.	Deployment-specific bugs and AI behavior need time for debugging and polish.
A06-07	Fast-track and safely overlap remaining sprint work.	The team needs to recover schedule and preserve release time.
A06-08	Do not prioritize separate sprint-demo recording work unless the submission requirement is confirmed.	Use limited time on assessed deliverables and release readiness.
A06-09	Keep technical/architectural documentation current as changes are made.	Documentation is part of the capstone evidence, not a final-week reconstruction task.
A06-10	Bring Trello into alignment with PR #34, actual owners, and real completion status.	The board is part of the project record and must describe reality.

6. Combined Project Status After August 6

Area	What had been achieved	What still required attention
Database / schema	Schema work and Alembic migration direction were in place; persistence foundations were substantially established.	Continue controlled schema evolution through reviewed migrations and keep API/schema contracts stable.
Authentication / frontend	Login and core React flows were working/progressing; wireframes remained the agreed visual reference.	Finish remaining user flows, align all pages to the design artifacts, and verify behavior against production services.
AI / analysis	AI functionality was important enough to be treated as the centerpiece of final integration testing.	Finalize/verify the provider behavior and ensure the results returned by the backend are correct, explainable, and rendered properly.
History	Local application was broadly functioning.	History remained an explicit missing/incomplete end-to-end flow and needed implementation plus regression testing.
PR workflow	A large number of PRs had been reviewed/closed compared with earlier in the project.	Reduce time between approval and merge; avoid stale branches and mass merges.
Architecture	PR #34 documented important alignment issues and was treated as ready to close.	Make code, Trello, README/ADR material, and future implementation follow the same decisions.
Local integration	Team members reported the application working well locally and had started repeated manual testing.	Repeat the full flow after remaining merges and again in production.
Deployment	A concrete AWS direction had been agreed and Samar had already completed part of the path.	Finish GitHub Actions deployment, OIDC, ECR, EC2 configuration, environment variables, domain, and smoke tests.

Area	What had been achieved	What still required attention
Trello / planning	The board still provided the sprint structure and showed active work.	Correct stale cards, ownership, completion states, and PR #34-related wording.
Documentation	The docs repository structure and architecture-review practice were in active use.	Continue recording decisions now; complete user-facing documentation later when behavior stabilizes.
Schedule	The team acknowledged the problem openly and began discussing recovery.	Complete the current critical work quickly and protect time for final integration, deployment and submission.

7. Production Deployment Plan Agreed on August 6

The deployment conversation was detailed enough to record the intended sequence separately. This is the most concrete production plan captured in the August 6 meeting.

Step	Activity	Primary owner / involvement
1	Finish and merge the remaining release-critical feature work through the normal PR and CI process.	Entire team
2	Extend GitHub Actions from pre-merge quality checks into the deployment workflow.	Samar, with Ranga support
3	Build the application Docker image in the CI/CD workflow.	GitHub Actions / DevOps
4	Authenticate GitHub Actions to AWS using OIDC.	Samar / DevOps
5	Push the built image to Amazon ECR.	GitHub Actions / AWS
6	Deploy/pull the approved image onto the EC2 production environment.	Samar / DevOps
7	Configure required production environment variables and runtime settings.	DevOps + relevant backend owners
8	Connect Ahmed's custom domain to the public production deployment.	Ahmed
9	Run production integration/smoke testing, with particular attention to AI output, persistence, navigation, History, and frontend rendering.	Ahmed + team
10	Fix production-only defects, update docs, and repeat the test until the release candidate is stable.	Entire team

8. Consolidated Action Items by Team Member

8.1 Ahmed Al-Mandalawi

- Continue end-to-end integration testing as the remaining PRs are merged, with special attention to functionality rather than only appearance.
- Verify the AI analysis path and make sure the output shown in the frontend is the output the team intends to demonstrate.

- Retest the History flow after implementation because it was the clearest remaining end-to-end gap identified on August 6.
- Connect the custom domain as soon as Samar's production environment and required variables are ready.
- Perform production smoke testing early enough to allow time for debugging, regression testing, and polish.
- Continue maintaining meeting records and preserving the decision and test evidence needed for the capstone documentation.

8.2 Ranga Nyamadzawo

- Continue backend/schema work using Alembic and the agreed contracts.
- Keep repository guardrails aligned with the team's real architecture and protect shared schema/API contracts from silent drift.
- Close the loop on PR #34 and ensure future architecture changes are documented rather than remaining informal.
- Encourage prompt merging of approved PRs and assist with conflict/regression resolution.
- Support Samar on the AWS deployment work and review her PRs quickly when tagged.

8.3 Mulima Chibuye

- Maintain the sprint recovery plan and keep visibility on the current eleven sprint cards and the next group of work.
- Update the Trello cards referenced by PR #34 so their wording matches the decisions the team actually made.
- Clean up cards that are missing owners, sitting in the wrong column, or already completed but not moved to Done.
- Use PR references to improve traceability between Trello planning and GitHub implementation.
- Continue facilitating weekly sprint/recovery discussions and escalate unassigned or blocked work early.

8.4 Adrian Muchatibaya

- Continue React implementation using the approved wireframes as the visual source of truth.
- Raise frontend work through feature branches and PRs rather than delaying because nearby work is happening in parallel.
- Continue connecting frontend pages to completed backend routes and verify the full user flow as each route becomes available.
- Share and maintain the existing Render skeleton URL as a useful interim artifact while AWS production is being prepared.

8.5 Samar El Ghandour

- Complete the AWS deployment proof of concept and turn it into a repeatable team deployment pipeline.
- Extend GitHub Actions so the workflow can build the Docker image and perform the AWS deployment path.
- Configure GitHub-to-AWS OIDC authentication, ECR image delivery, and EC2 deployment.
- Raise deployment changes incrementally through PRs and tag Ranga or other relevant reviewers so feedback does not become a new bottleneck.
- Make the production environment available early enough for Ahmed to connect the domain and start public-environment testing.

8.6 Entire team

- Review and merge approved PRs promptly.
- When a second review is desired, ask for it explicitly in the team chat rather than leaving the PR idle.
- Keep Trello cards assigned, current, and in the correct column.
- Document architectural or design decisions in the repository as part of the same work that implements them.
- Fast-track remaining release-critical work and overlap independent tasks where safe.
- Protect enough time for production deployment, regression testing, documentation, and the final presentation rather than allowing feature work to consume the entire schedule.

9. Risks, Blockers and Delivery Concerns

Risk / issue	Evidence in the two meetings	Potential effect	Response
Schedule compression	Team described itself as behind on July 22 and again on August 6; large numbers of cards remained.	Little time for deployment, bug fixing, docs and presentation work.	Fast-track critical work, overlap safe tasks, and protect release time.
PR review/merge latency	Approved PRs remained open long enough to conflict with newer main changes.	Rework, conflicts and late regressions.	Merge approved work promptly; explicitly request second reviews.
Architecture drift	Shared-contract guardrails were discussed in July; PR #34 later documented wider conflicts.	Different contributors may build against incompatible assumptions.	Use PR #34, docs, repository development guidance and updated Trello cards as a single alignment mechanism.
History incomplete	August local testing found the History path not fully implemented/connected.	A core authenticated-user feature remains broken.	Complete it and include it in local and production regression testing.
Production deployment learning curve	Samar was implementing the AWS route for the first time and was still experimenting.	Infrastructure problems could consume the final project window.	Start now, work incrementally, use Ranga's support and test early.
Cloud cost uncertainty	Render billing caused concern; AWS free-tier/ECR limits were not verified in the meeting.	Unexpected charges or resource limits.	Verify the actual account/region limits during deployment setup.
AI/model behavior	Team expected potential model complications and stressed correct output as central to the final demo.	Unreliable or confusing analysis at presentation time.	Leave time for production AI testing, validation and fallback decisions.
Trello drift	Cards lacked owners/status and did not fully reflect PR #34 decisions.	Duplicate work and weak project evidence.	Mulima cleans the board; everyone updates cards immediately.
Documentation debt	Team explicitly said documentation would be evaluated and should be built as the project evolves.	Final submission may not explain or match the actual implementation.	Create ADRs/reviews/minutes alongside implementation; finish user docs when stable.
Competing academic workload	Members still had remaining MSSE courses due around August 12.	Reduced capacity during a critical delivery period.	Communicate capacity early and keep ownership visible.

10. Open Questions Not Fully Resolved in the Meetings

- What exact production PostgreSQL hosting arrangement will accompany EC2? The August 6 discussion established the application-image path but did not settle the final database hosting, backup, or recovery design.
- What are the verified AWS free-tier and ECR limits for the actual account and region? The meeting contained several informal estimates but no confirmed billing reference.

- What exact DNS/TLS configuration will be used when Ahmed connects the custom domain? Ownership of the domain step is clear, but the detailed production DNS/HTTPS procedure was not discussed.
- What is the final LLM/provider configuration for production, and what acceptance criteria will determine that the model is reliable enough for the demonstration?
- What is the exact completion date for the final group of seven Trello cards mentioned after the August 26 work? The source transcript does not capture the date.
- Will the existing Render skeleton remain only as an interim artifact once AWS is live, or will it be kept as a fallback/test environment?
- Which sprint demo recordings are formally required by Quantic, and which are optional internal evidence? The team decided not to spend time on extra recordings unless they are required, but the external requirement was not confirmed in the meeting.

11. Project Evidence to Preserve

Both meetings reinforced that the capstone is being assessed not only as a final website but as an engineering project. The records below should therefore be kept because they demonstrate planning, review, traceability, testing, and decision-making:

- Trello history showing sprint assignment, ownership, movement through workflow states, and completed cards.
- PR #34, including its architecture findings, comments, approvals, and resulting changes to cards or documentation.
- Database/schema and Alembic migration pull requests.
- Repository development guardrails explaining how shared contracts and architecture are protected.
- Wireframes and the frontend PRs that implement them.
- GitHub Actions workflows before and after deployment automation is added.
- AWS deployment notes covering OIDC, ECR, EC2, environment variables, and the actual release procedure.
- Local and production integration/smoke-test evidence, especially for registration/login, analysis, History, persistence, and AI rendering.
- Meeting minutes, ADRs, architecture reviews, README updates, and the eventual user-facing instructions.
- Any screenshots or recordings that genuinely demonstrate working increments and add evidence rather than duplicate existing material.

12. Recommended Agenda for the Next Weekly Alignment

10. Confirm PR #34 has been merged/closed and verify that its related Trello cards now match the agreed architecture.
11. Review every current-sprint card for real owner, real status, blocker, and whether it can be moved to Done.
12. Confirm whether the History feature is now implemented and testable end to end.
13. Get a live deployment update from Samar: GitHub Actions, OIDC, ECR, EC2, current blocker, and date of first usable public environment.
14. Confirm who owns production database configuration and how migrations/backups will be handled.
15. Confirm when Ahmed can connect the domain and begin production smoke testing.
16. Review the production LLM/provider choice and identify any output-quality issues that still need testing.

17. Check CI after the latest merges and make sure there is no new queue of approved-but-unmerged PRs.
18. Review documentation/ADR changes and ensure Trello, README material, diagrams, and implementation still agree.
19. Agree on the smallest remaining release-critical scope and prevent nonessential work from displacing deployment, testing, documentation, and presentation preparation.

13. Closing Summary

The July 22 stand-up captured a project that had finally moved beyond setup but was still assembling its core foundations. The schema and migration approach were taking shape, login/frontend work was becoming testable, wireframes were being treated as a real design contract, and the team was beginning to formalize how implementation work should respect shared architecture and regression safety. The team already knew it was behind and was asking members to keep Trello accurate so that parallel work could continue without duplication.

The August 6 meeting shows how quickly the nature of the work had changed. The local application was now broadly functional, many PRs had moved, and the project was close enough to completion that deployment and release process had become the dominant risk. The team identified slow PR merging, architecture/planning drift, a still-incomplete History path, Trello inaccuracies, and a shrinking schedule as issues that could no longer be treated as minor process problems.

The most consequential August 6 technical decision was the move toward an AWS production workflow in which GitHub Actions would build the application image, authenticate to AWS using OIDC, publish the image to ECR, and deploy it to EC2. This records the team's direction at that point in the project; later implementation details remained subject to the repository and deployment work that followed.

Across both meetings, the central delivery principle became consistency and visibility. The code, API contracts, architecture notes, pull requests, Trello cards, tests, and final documentation all need to describe the same system. The remaining work is therefore not simply to finish individual features. It is to integrate them continuously, merge and test them promptly, document the decisions that shaped them, deploy a stable production version, and leave enough time to prove that the final TrustAI Marketplace application works as one coherent product.